

AI Interview Prep Platform - Technical Implementation & Architecture Guide

1. System Overview

The AI Interview Prep Platform is a full-stack application designed to help candidates prepare for technical and behavioral interviews. It leverages large language models (LLMs) to generate customized study plans and conduct interactive, voice-enabled mock interviews.

High-Level Architecture:

- **Frontend:** A modern React application built with Vite and Tailwind CSS. It provides a polished, glassmorphism-inspired UI for user interaction.
- **Backend:** A robust FastAPI server written in Python. It handles API requests, orchestrates AI agents using LangChain, and manages context retrieval.
- **Vector Database:** Qdrant is used for Retrieval-Augmented Generation (RAG). It stores embedded chunks of real interview experiences (e.g., LeetCode discussion scrapes) so the AI can reference realistic interview questions.
- **AI Inference Engine:** NVIDIA NIM (via `langchain_nvidia_ai_endpoints`) is used to run state-of-the-art open-weights models (like Llama 3.1 and DeepSeek) with incredibly low latency.

2. AI Models & Automatic Fallback Mechanism

One of the most critical aspects of the backend is the robust handling of LLM inference and API rate limits.

The backend relies on the **NVIDIA Developer API**, which enforces strict rate limits on free-tier usage (e.g., 50 requests max for certain models). To ensure the platform never breaks or returns `503 Service Unavailable` errors during a live mock interview, an **Automatic Model Fallback System** was implemented.

How the Fallback Works: The backend maintains a cascading list of highly capable models:

1. `meta/llama-3.1-8b-instruct` (Fast, high rate limits)
2. `google/gemma-2b` (Extremely lightweight)
3. `deepseek-ai/deepseek-v4-flash` (Advanced reasoning)
4. `meta/llama-3.1-70b-instruct` (Massive context, slower generation)

When a user requests a study plan or submits a chat message, the backend attempts to use the first model. If the NVIDIA API returns a `ResourceExhausted` (Rate Limit) or `Timeout` error, the backend instantly catches the exception and flawlessly retries the exact same prompt with the next model in the list. This ensures 100% uptime without the user ever noticing a failure.

3. Frontend Architecture

The frontend is a Single Page Application (SPA) built for high performance and premium aesthetics.

Key Technologies: - **React 18 & Vite:** For fast hot-reloading and optimized production builds. - **Tailwind CSS & clsx:** Utility-first styling for complex, responsive, and dynamic UI elements (like the glassmorphism effects). - **Lucide React:** For sleek, scalable vector icons. - **Web Speech API (Text-to-Speech):** A native browser integration that allows the AI interviewer to "speak" its dialogue out loud. The system intelligently strips out markdown and internal `<think>` blocks before speaking, providing a natural voice experience.

4. Backend Architecture

The backend is completely decoupled from the frontend, communicating exclusively via RESTful JSON APIs.

Key Technologies: - **FastAPI:** A high-performance Python web framework for building asynchronous APIs. - **LangChain:** An orchestration framework used to build complex prompt chains and manage the flow of data between the user, the vector database, and the LLM. - **Qdrant:** A containerized vector similarity search engine running via Docker Compose. - **Sentence Transformers:** Used locally via `HuggingFaceEmbeddings` (specifically `all-MiniLM-L6-v2`) to turn text into vector embeddings without relying on paid external APIs.

5. Detailed File Structure & Explanation

Below is an exhaustive breakdown of every critical file in the project and its exact purpose.

5.1 Root Directory (/ai-interview-prep)

- `docker-compose.yml` : The orchestration file for the backend. It spins up three isolated Docker containers: the FastAPI backend server, the Qdrant vector database, and a PostgreSQL database (reserved for future user data storage).

5.2 Frontend Directory (/frontend)

- `package.json` & `vite.config.ts` : Configuration files defining npm dependencies, build scripts, and the Vite bundler settings.
- `index.html` : The root HTML file that mounts the React application.
- `src/main.tsx` : The entry point for the React application. It wraps the app in React Router for navigation.
- `src/index.css` : The global stylesheet containing Tailwind directives and custom CSS variables for the glassmorphism theme and dark mode colors.
- `src/App.tsx` : The main layout component. It defines the navigation sidebar and the React Router `<Routes>` configuration.
- `src/pages/Dashboard.tsx` : The landing page UI showing mock metrics and quick action buttons.

- `src/pages/CompanyPrep.tsx` : The UI for generating customized 14-day study plans. It collects the user's target company, role, and timeline, then sends a POST request to the backend `/study-plan` endpoint.
- `src/pages/MockInterview.tsx` : The interactive interview interface. It features simulated camera/microphone feeds, a live chat transcript, and the integrated Text-to-Speech engine. It communicates with the backend `/chat` endpoint using the `[MOCK INTERVIEW MODE]` prefix to trigger the strict interviewer AI persona.

5.3 Backend Directory (`/backend`)

- `requirements.txt` : Defines all Python dependencies (FastAPI, LangChain, Uvicorn, Qdrant Client, etc.).
- `Dockerfile` : Instructions for building the Python backend image. It installs the requirements and copies the application code into the container.
- `main.py` : The entry point for the FastAPI application. It configures CORS middleware (allowing the frontend to communicate with it) and mounts the API routers.

5.3.1 Backend Application Code (`/backend/app`)

- `core/config.py` : Manages environment variables and configuration settings using Pydantic (e.g., loading the `NVIDIA_API_KEY`).
- `api/endpoints/chat.py` : The API router defining the REST endpoints (`POST /api/v1/ai/chat` and `POST /api/v1/ai/study-plan`). It processes incoming requests, strips special prefixes (like `[MOCK INTERVIEW MODE]`), and routes them to the appropriate LangChain agents.
- `ai/retriever.py` : Connects to the Qdrant vector database. It initializes the `HuggingFaceEmbeddings` model and provides a function (`get_retriever`) to fetch contextually relevant interview experiences based on the user's query and metadata filters (company/role).
- `ai/agents/interview_agent.py` : The core logic for the Mock Interview chat. It contains:
 - The `FALLBACK_MODELS` list and the automatic retry loop.
 - The `system_prompt` (for general mentoring).
 - The `mock_interviewer_prompt` (which explicitly instructs the AI to ask only one question at a time, wait for a response, and evaluate the candidate).
 - The execution logic inside a `ThreadPoolExecutor` to enforce strict timeouts and prevent the server from hanging indefinitely.
- `ai/agents/study_plan_agent.py` : The logic for generating the 14-day study roadmap. Similar to the interview agent, it utilizes the `FALLBACK_MODELS` list, strict `ThreadPool` timeouts, and a specialized `study_plan_prompt` designed to output concise, highly structured Markdown.

Conclusion

The AI Interview Prep Platform successfully integrates modern web aesthetics with cutting-edge open-weights AI models. By implementing local vector embeddings and robust fallback mechanisms

on top of the NVIDIA NIM infrastructure, the system provides a highly responsive, intelligent, and completely free alternative to expensive proprietary AI coaching services.